

Relatório Técnico de Desenvolvimento e Avaliação de Desempenho: Arquitetura de Monitoramento Preditivo de QoS Baseada em Aprendizado de Máquina

Pedro Henrique Cardoso Teixeira de Paula, Hugo Antonio Fernandes dos Santos

Tecnólogo em Redes de Computadores – Instituto Federal da Paraíba (IFPB)

`pedro.cardoso@academico.ifpb.edu.br`, `antonio.hugo@academico.ifpb.edu.br`

Abstract

This report describes the complete trajectory of the conception, implementation, and homologation of a predictive Quality of Service (QoS) monitoring ecosystem oriented towards video streaming environments (such as YouTube and Netflix). The developed architecture combines a containerized infrastructure using enterprise tools (Zabbix and Grafana) with an Artificial Intelligence layer based on the Random Forest algorithm. The document details the project's evolution in two major phases, describing the statistical calculations, the transition to real-time analysis, and validation in Chaos Engineering scenarios — including physical disconnection, flood attacks (DDoS), and stealth interception (Man-in-the-Middle).

Resumo

Este relatório descreve a trajetória completa de concepção, implementação e homologação de um ecossistema de monitoramento preditivo de Qualidade de Serviço (QoS) orientado a ambientes de streaming de vídeo (como YouTube e Netflix). A arquitetura desenvolvida une uma infraestrutura containerizada utilizando ferramentas corporativas (Zabbix e Grafana) a uma camada de Inteligência Artificial baseada no algoritmo *Random Forest*. O documento detalha a evolução do projeto em duas grandes fases, descrevendo os cálculos estatísticos, a transição para análise em tempo real e a validação em cenários de Engenharia do Caos - incluindo rompimento físico, ataques de inundação (DDoS) e interceptação furtiva (*Man-in-the-Middle*).

1 Introdução e Objetivos

O crescimento gigantesco do tráfego de vídeos na internet cria um desafio diário para os administradores de redes. Aplicativos modernos de streaming utilizam um protocolo chamado DASH (*Dynamic Adaptive Streaming over HTTP*). Em termos simples, o DASH tenta contornar a lentidão da internet baixando blocos enormes de vídeo de uma só vez e guardando-os na memória do dispositivo (o chamado *buffer*, que atua como uma "reserva de emergência").

Embora o buffer evite que o vídeo trave imediatamente quando a internet oscila, ele cria um problema para os técnicos: ele "mascara" falhas graves na rede. Quando o vídeo

finalmente congela na tela do usuário, significa que o problema na infraestrutura já estava acontecendo há vários segundos e a reserva de emergência acabou.

Objetivo: Este projeto teve como objetivo criar um sistema de Inteligência Artificial capaz de atuar como um "radar preditivo". A meta é analisar o comportamento matemático da placa de rede para detectar essas falhas silenciosas na infraestrutura *antes* que o buffer do usuário se esgote, permitindo que a equipe de TI atue de forma proativa.

2 Arquitetura da Infraestrutura

Para garantir que o sistema fosse leve e fácil de replicar, toda a base foi construída utilizando a tecnologia Docker (que empacota softwares em "contêineres" isolados). A topologia é composta por:

- **Banco de Dados (MySQL):** O "cofre" do projeto, responsável por guardar todas as métricas e configurações de forma otimizada.
- **Motor de Monitoramento (Zabbix Server):** O coração da operação. Ele recebe os alertas da Inteligência Artificial em tempo real.
- **Interface e Visualização (Grafana):** O painel de controle visual. Ele se conecta ao Zabbix e traduz os dados complexos em gráficos coloridos e intuitivos, criando um verdadeiro Centro de Operações de Rede (NOC).

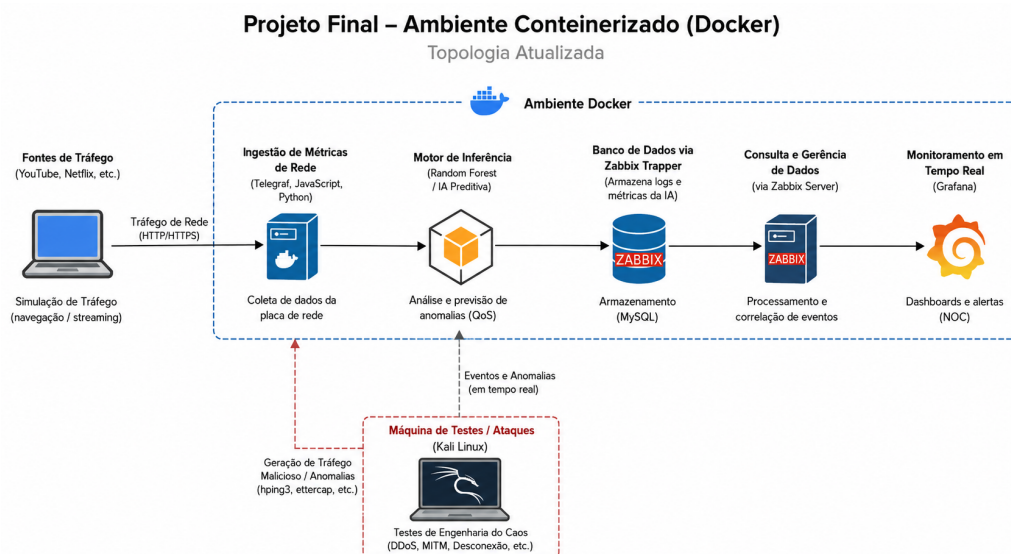


Figura 1: Topologia Atualizada do Ambiente de Infraestrutura e Monitoramento.

Nota Técnica de Resolução de Conflitos: Durante a implantação, superamos um desafio de permissões do sistema operacional Linux que impedia o Grafana de salvar seus painéis. Isso foi resolvido ajustando a propriedade da pasta no nível do kernel para o usuário correto:

```
sudo chown -R 472:472 grafana_data/
docker compose restart grafana
```

3 Fase 1: Concepção da IA e O Cérebro Matemático

A primeira fase focou em ensinar a Inteligência Artificial a reconhecer o que é uma rede "saúdável" e o que é uma rede "doente". Utilizamos o algoritmo *Random Forest* (Floresta

Aleatória), que cria centenas de árvores de decisão para chegar a um veredito. Através de análises estatísticas, observamos que o primeiro dataset não era rotulado, então tivemos de criar um script que faria a coletânea das métricas que iríamos usar no projeto e também fizesse a rotulação do que seria uma rede estável (status 0) e o que seria uma rede com possível degradação (status 1) e o arquivo criado foi o "pre-processamento.py", definimos que qualquer atraso (latência) superior a **45 milissegundos** seria considerado o limiar crítico de gargalo, e as métricas separadas do dataset bruto foram "Tempo, Tamanho, ARTT" e a nova coluna adicionada para a rotulação foi a "Status" e o dataset processado ao final do arquivo teria o nome "dataset-o nome da plataforma escolhida-processado.csv"

O Treinamento da IA: Para treinar a IA utilizando o dataset gerado pelo script anterior nos criamos um arquivo python "treinar-modelo.py", que cria um modelo Random Forest que lê o dataset gerado e separa o "tamanho dos pacotes" e o "ARTT" e treina a IA baseado na coluna Status que foi gerada anteriormente treina ela para seguir os parametros que foram dados. Ao final disso, pelo script se perder ao fim de seu código, a forma que encontramos de salvar o estado da IA foi utilizando a biblioteca joblib, ela "congelou" o estado da IA que estava armazenada na memória RAM e gerou o arquivo "modelo-ia-qos.pkl".

Data Replay: Para ver a IA funcionando depois de treinarmos, criamos o script "agente-streaming.py" que carregava o modelo da IA que foi gerado utilizando o comando "modelo-rf = joblib.load('modelo-ia-qos.pkl')", juntamente com algum csv pertencente ao dataset, fazendo a separação das linhas que seriam usadas e eram enviadas para a IA analisar utilizando o comando "previsao = modelo-rf.predict" e como saída a IA enviada 0 ou 1, a saída era armazenada e enviada juntamente com os dados do dataset separados para o zabbix e o mesmo abastecia os gráficos do grafana.

A **Figura 2** ilustra perfeitamente essa fronteira matemática dos 3 scripts diferentes de cada plataforma do dataset sendo analisados pela IA simultaneamente. Independentemente do tamanho do pacote de dados (representado no eixo horizontal em escala logarítmica), se a latência ultrapassar a linha crítica de 45ms (linha tracejada), a IA classifica matematicamente o evento como uma anomalia na zona vermelha superior.

Conforme demonstrado na **Figura 3**, essa imagem mostra a análise da IA para qual as plataformas de forma separada uma por uma (Netflix, YouTube e Amazon) simultaneamente. A IA consegue isolar as degradações específicas de cada plataforma sem gerar uma quantidade significativa de falsos positivos, e enviando os alertas.

4 Fase 2: A Sonda Live e a Matemática Avançada

Na Fase 2, o projeto abandonou os testes utilizando o primeiro dataset pois seus dados eram muito limitados ao tamanho dos pacotes e ARTT, nos encontramos um novo dataset com novas métricas, também não rotulado mas organizado por tipos de tráfego, e seguimos

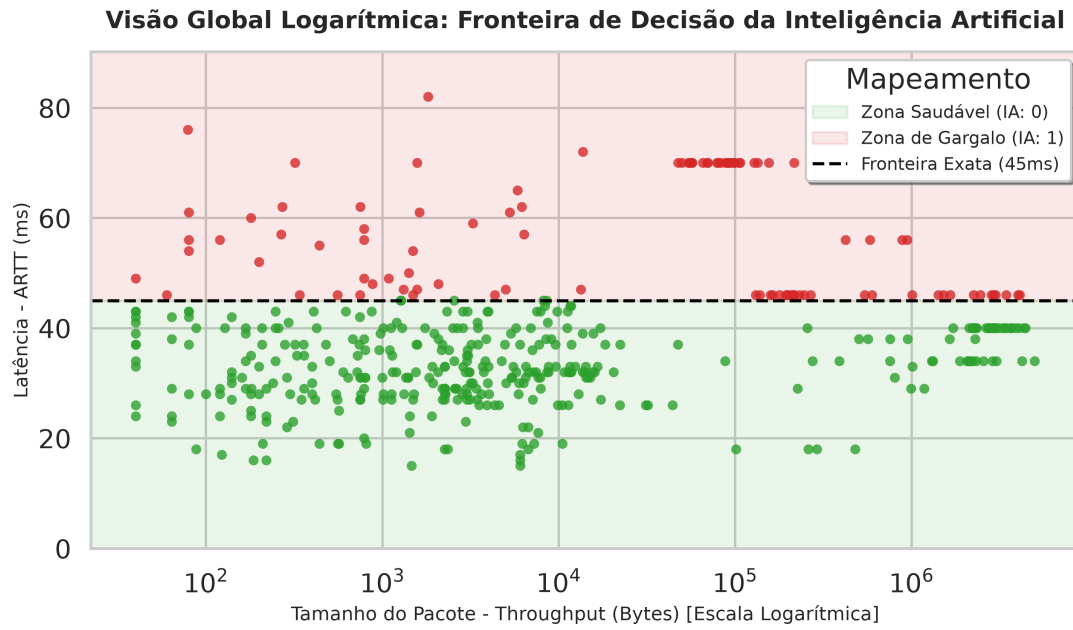


Figura 2: Visão Global Logarítmica: Fronteira de Decisão do Algoritmo Random Forest.

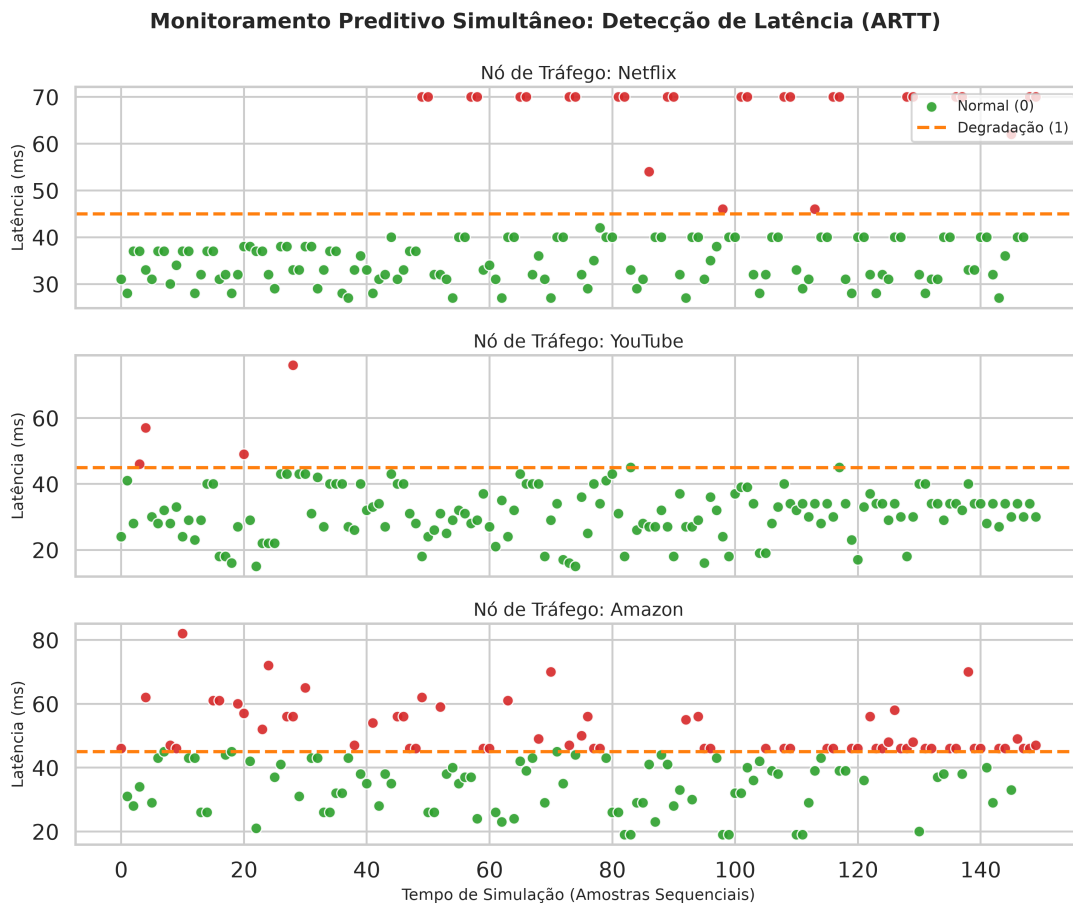


Figura 3: Monitoramento Preditivo Simultâneo (Linha do Tempo) de múltiplos terminais.

a mesma lógica anterior de refinamento, porém como era um arquivo csv único e grande parte dele eram números muito próximos em sua grande maioria de tráfegos estáveis, decidimos dividi-lo em 4 partes utilizando o Script "refinar-dataset" utilizando as métricas "fluxo de bytes por segundo", "fluxo de pacotes por segundo", "Tempo Médio entre chegadas", "Jitter", "Tempo de download do chunk" e o "Tempo de descanso do player": Uma delas usariamos para treinar a segunda versão da IA para o "Status 0" usando os dados reais do dataset; A segunda parte nos modificamos os dados para simular uma degradação de rede alterando as métricas acima; A terceira partes nos usamos para simular uma queda de rede, zerando todas as métricas e aumentando o "Tempo de Descanso", porém para diferenciar o consumo de buffer de uma real queda de rede, nos inferimos que 5 segundos de descanso sem passar tráfego a IA devia classificar queda de rede (Status 2) e abaixo disso seria rede estável ou consumo de buffer do DASH (Status 0) E por fim a quarta e menor parte seria para testar a IA usando o que restou do dataset.

Ao final disso o cada parte do dataset modificado é concatenado em 1 e usado para treinar a IA, gerando os arquivo "dataset-streaming-v2.csv" e o "modelo-qos-v2.pkl".

```
(venv) hugo@hugo-B550M-K:~/Projeto-Avalia-o-de-Desempenho-en-Redes/projeto-qos-nl/Fase 2/archive$ ./venv/bin/pyth
n3 refinar_dataset.py
1. Lendo o tráfego bruto do Kaggle...
-> Encontradas 1204 amostras de Streaming perfeito.
2. Fabricando cenários de Rede (Normal, Gargalo, Buffer e Queda)...
3. Treinando o Modelo Random Forest Definitivo...

--- Boletim da IA ---
              precision    recall  f1-score   support

Limpo/Buffer (0)      0.93      0.98      0.95       129
Degradação (1)       0.97      0.87      0.92        76
Queda Real (2)       1.00      1.00      1.00         52

   accuracy            0.95            257
  macro avg           0.97            257
 weighted avg           0.95            257

Sucesso! O ficheiro 'modelo_qos_v2.pkl' agora diferencia Buffer de Queda de Rede.
(venv) hugo@hugo-B550M-K:~/Projeto-Avalia-o-de-Desempenho-en-Redes/projeto-qos-nl/Fase 2/archive$
```

Análise do Relatório de Classificação:

Esse relatório utiliza algumas métricas para medir o desempenho da Inteligência Artificial na classificação dos estados da rede.

Precision (Precisão): representa quantas vezes o modelo acertou quando previu uma determinada classe. Quanto maior a precisão, menor a quantidade de falsos positivos.

Recall (Sensibilidade): mostra quantas ocorrências reais de uma classe foram corretamente identificadas pelo modelo. Quanto maior o recall, menor a quantidade de falsos negativos.

F1-Score: é a média entre precisão e recall, servindo como uma métrica de equilíbrio entre ambas e resumindo o desempenho geral do modelo.

Support: indica a quantidade total de amostras presentes no conjunto de testes para cada classe.

Accuracy (Acurácia): representa o percentual total de acertos do modelo considerando todas as classes.

Macro Avg: calcula a média das métricas de todas as classes, atribuindo o mesmo peso para cada uma.

Weighted Avg: calcula a média das métricas considerando o peso de cada classe com base na quantidade de amostras.

Antes de ir para os testes reais nos atualizamos o arquivo anterior "agente-streaming" para carregar o novo arquivo da IA para analisar o novo dataset refinado com o script "refinar-dataset" juntamente com o primeiro dataset (

Youtube, Netflix e Amazon) e mantendo a mesma lógica do comando `"modelo-rf = joblib.load('modelo-qos-v2.pkl')"` para carregar a IA e vermos como se comporta, o resultado é possível ver no primeiro teste anexado ao final do documento.

5 Engenharia do Caos: Homologação e Testes de Estresse

Para provar o valor do monitoramento, aplicamos os conceitos de "Engenharia do Caos" a prática de quebrar a própria rede de propósito para ver como o sistema reage. Para isso criamos o Script `"agente-live.py"` utilizando o `scapy`, para ouvir a placa e rede de uma máquina real, o mesmo carrega a IA e faz um Sniff na placa de rede usando filtro `"TCP ou UDP"` na porta 443, ele separa as métricas (`"vazao"`, `"densidade"`, `"tempo-medio"`, `jitter`, `tempo-ativo`, `tempo-ocioso`) encontradas na escuta e volta a usar o comando `"previsao = modelo-rf.predict(dados-rede)"` para pedir a avaliação da IA, a mesma é armazenada e enviada para o `zabbix` juntamente com as métricas da interface e carregados no `grafana` para melhor visualização.

Utilizamos também uma VM *Kali Linux* para conduzir três cenários práticos contra uma transmissão ao vivo no YouTube.

5.1 Cenário A: Falha Estrutural Física (Queda de Cabo)

- **Ação:** O cabo de rede foi removido abruptamente durante o streaming em resolução máxima.
- **Resultado:** Como esperado, o vídeo na tela continuou rodando perfeitamente por vários segundos, consumindo a memória buffer. No entanto, a IA detectou a morte instantânea dos dados na camada física. Após a tolerância de 5 segundos, o painel do Grafana acionou o alerta máximo vermelho de Rede Inativa.

A **Figura 4** evidencia esse exato momento da ruptura física. A ociosidade da placa ultrapassa abruptamente o limite e o Jitter se dispersa na zona vermelha devido à destruição da cadência de pacotes.

5.2 Cenário B: Ataque de Inundação / DDoS (Estresse Bruto)

- **Ação:** Utilizando a ferramenta `hping3` no Kali Linux, disparamos milhares de pacotes falsos na porta 443 para estrangular a banda do roteador (*UDP Flood*).
- **Resultado:** A Sonda identificou uma explosão na densidade de pacotes e uma queda irreal no Jitter (pois o tráfego tornou-se um fluxo robótico ininterrupto). A IA diagnosticou instantaneamente a anomalia, cravando o status de Degradação muito antes da rede colapsar.

5.3 Cenário C: O Estrangulador Furtivo (Man-in-the-Middle)

- **Ação:** O teste mais complexo. O Kali Linux interceptou silenciosamente a conexão entre o computador e o roteador, injetando matematicamente 80ms de atraso e 30ms de variação (Jitter) nos pacotes do YouTube.

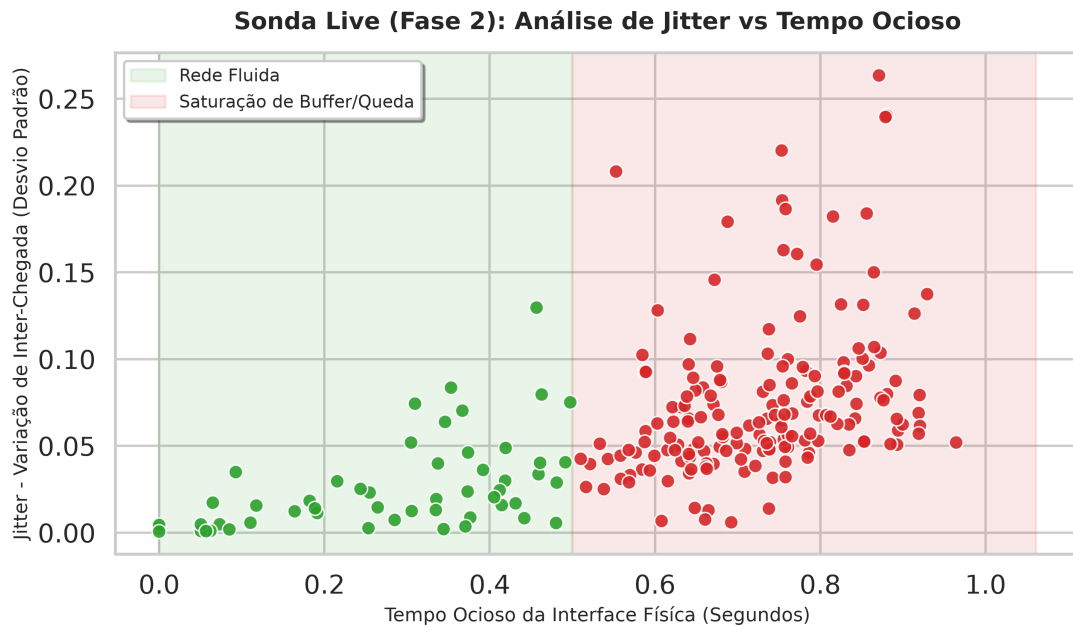


Figura 4: Sonda Live: Dispersão de Jitter vs Tempo Ocioso durante a falha estrutural.

- **Resultado:** Este teste provou o principal argumento do projeto. A máquina alvo não perdeu a conexão com a internet e o vídeo não travou em nenhum momento. Contudo, a IA calculou o atraso milimétrico oculto nos pacotes e alertou no painel do Zabbix a perda preditiva da Qualidade de Serviço (QoS).

OBS: Ambos os cenários A e B podem ser vistos no vídeo do segundo teste, e o cenário C pode ser visto no terceiro teste, todos anexados ao final do documento.

6 Conclusão

O projeto alcançou sucesso em demonstrar como o Aprendizado de Máquina pode revolucionar a gerência de redes (AIOps). Os testes provaram que monitorar o comportamento estatístico diretamente na placa de rede supera a ilusão de estabilidade gerada pelos aplicativos de vídeo. Nossa arquitetura mostrou-se capaz de alertar o Centro de Operações (NOC) sobre degradações severas, ataques de inundação e sequestros de tráfego de forma preditiva.

Vídeos de Demonstração (Testes de Homologação):

Os vídeos com as execuções práticas dos testes e a simulação das falhas (Engenharia do Caos) podem ser acessados nos seguintes links do Google Drive:

- **Vídeo 1: [1º teste-Topologia]**
Link: https://drive.google.com/file/d/1n4IJw6W22nGJbEJmAnfnIsa0WifXrxE5/view?usp=drive_link
- **Vídeo 2: [2º teste-DDOS]**
Link: https://drive.google.com/file/d/1oiJ3AVzoq01TmEs-Wvx_c90HRw8v5JdX/view?usp=drive_link

- **Vídeo 3: [3° teste-MantheMiddle]**

Link: https://drive.google.com/file/d/1h67B_bJKoMnyhHlKVyy0Had0PJEqYsL/view?usp=drive_link

Repositório do Projeto: Todos os códigos-fonte e configurações mencionadas estão disponíveis publicamente no repositório:

<https://github.com/0hugoantonio83/Projeto-Avalia-o-de-Desempenho-em-Redes>

Bases de Dados (Datasets) de Referência:

- **Aggregated Gaming e Video Streaming Traffic for 5G**

Autor: Ahmed Hassanein

Link: <https://www.kaggle.com/datasets/ahassanein/aggregated-gaming-and-video-streaming-traffic-for-5g>

- **VPN and Non-VPN Application Traffic (CIC-VPN2016)**

Autor: Krish Agarwal

Link: <https://www.kaggle.com/datasets/noobbcoder2/vpn-and-non-vpn-application-traffic-cic-vpn2016>